

System Architecture Document

Backend API, Database Schema & Data Collection

Tech Stack

Backend: Node.js + Express

Database: MongoDB

Auth: Session-based (username/email/password)

Data Collection: POST /api/collect — centralized collector endpoint

Version 0.8 — Draft

1. System Overview

This document defines the backend architecture for the application monitoring and health dashboard system. The system consists of an Express/Node.js server, a MongoDB database, and a data collection pipeline inspired by the CS 135 collector pattern.

The collector pattern works as follows: client applications send all instrumentation events to a single POST /api/collect endpoint. That endpoint normalizes and persists the payload into MongoDB. Separate read endpoints then serve the stored data to the dashboard frontend.

2. API Endpoints

Endpoints are split into two categories: data API routes (prefixed /api/) and page routes (returning rendered HTML or SPA entry points).

2.1 Authentication Routes

Method	Path	Description	Auth
POST	/api/signup	Register a new user account (username, email, password)	Public

Method	Path	Description	Auth
POST	/api/login	Authenticate and create a session (email, password)	Public
POST	/api/logout	Invalidate current session	Required
PUT	/api/update_account	Update user account information like username, password, and email	Required
DELETE	/api/delete_account	Delete user account	Required
GET	/api/me	Return current authenticated user profile	Required

2.2 Data Collection Route

All instrumentation from client applications flows through a single endpoint. The payload type field determines how the event is categorized and stored.

Method	Path	Description	Auth
POST	/api/collect	Ingest an event (error, performance, feedback, release)	Required

Accepted Payload Types

The type field in the request body must be one of the following values:

- error — JavaScript exceptions, unhandled promise rejections, caught errors
- performance — page load times, API response times, memory snapshots
- feedback — user-submitted ratings, comments, thumbs up/down signals
- release — deployment events, version bumps, rollback markers

Example Payload

```
{ "appId": "abc123", "type": "error", "message": "Uncaught TypeError", "stack": "...",
"url": "/dashboard", "userId": "u_99" }
```

2.3 Data Retrieval Routes

These routes are consumed by the dashboard frontend to render charts, tables, and health indicators. All require authentication. The :id parameter refers to the application (app) ID.

Method	Path	Description	Auth
GET	/api/performance/:id	Retrieve performance metrics for a given app	Required
GET	/api/bugs/:id	Retrieve error/bug reports for a given app	Required
GET	/api/feedback/:id	Retrieve user feedback signals for a given app	Required

Method	Path	Description	Auth
GET	/api/releases/:id	Retrieve release and version history for a given app	Required
GET	/api/apps	List all apps associated with the authenticated user	Required

Common Query Parameters

The retrieval endpoints should support the following optional query parameters (exact names TBD with frontend team):

- from / to — ISO 8601 date range filter
- limit — max number of records to return (default: 100)
- page — pagination offset
- severity — filter by severity level (for bugs endpoint)

2.4 Page Routes

These routes serve the frontend pages. They return rendered HTML or the SPA shell depending on the chosen rendering strategy.

Method	Path	Description	Auth
GET	/login	Login page	Public
GET	/signup	Signup / registration page	Public
GET	/home	User home / app list dashboard	Required
GET	/app/:id	App-specific health dashboard	Required

3. Database Schema (MongoDB)

All collections live in a single MongoDB database. ObjectId fields are automatically generated by MongoDB. Timestamps use ISO 8601 strings or MongoDB Date objects.

3.1 users

Stores account credentials and profile information.

Field	Type	Required	Notes
_id	ObjectId	Auto	Primary key
username	String	Yes	Unique, indexed
email	String	Yes	Unique, indexed, lowercase
passwordHash	String	Yes	bcrypt hash — never store plaintext
createdAt	Date	Auto	Set on document creation
updatedAt	Date	Auto	Updated on every save

3.2 apps

Represents a monitored application. A user can own multiple apps.

Field	Type	Required	Notes
_id	ObjectId	Auto	Primary key, used as :id in routes
ownerId	ObjectId	Yes	Ref: users._id
name	String	Yes	Display name of the application
apiKey	String	Auto	Generated secret for POST /api/collect auth
createdAt	Date	Auto	
updatedAt	Date	Auto	

3.3 events (collector store)

The canonical collection written to by POST /api/collect. All four event types (error, performance, feedback, release) are stored here in a polymorphic pattern. Type-specific fields are stored in a metadata sub-document.

Field	Type	Required	Notes
_id	ObjectId	Auto	Primary key
appId	ObjectId	Yes	Ref: apps._id — indexed
type	String (enum)	Yes	"error" "performance" "feedback" "release"
timestamp	Date	Yes	Client-reported or server-assigned on receipt
url	String	No	Page/route where event occurred
metadata	Object	No	Type-specific payload (see below)
receivedAt	Date	Auto	Server timestamp of ingestion

metadata sub-document by type

The metadata field is a flexible object. Below are the expected shapes per event type along with potential data elements:

type: "error"

Field	Type	Description
message	String	Error message text
stack	String	Stack trace
errorType	String	TypeError, RangeError, etc.
severity	String	"low" "medium" "high" "critical"

type: "performance"

Field	Type	Description
loadTimeMs	Number	Full page load time in milliseconds
ttfbMs	Number	Time to first byte (ms)
apiEndpoint	String	If API call, the endpoint measured
apiLatencyMs	Number	API round-trip time (ms)
memoryMB	Number	JS heap usage in MB at time of snapshot

type: "feedback"

Field	Type	Description
rating	Number (1-5)	Numeric star rating
sentiment	String	"positive" "neutral" "negative"
comment	String	Free-text user comment
featureName	String	Product feature this feedback relates to
thumbs	String	"up" "down" for simple binary signals

type: "release"

Field	Type	Description
version	String	Semver string, e.g. "2.4.1"
environment	String	"production" "staging" "development"
deployedBy	String	Username or CI system that triggered deploy
releaseNotes	String	Short changelog or commit message
rollback	Boolean	true if this event marks a rollback

3.4 sessions

Stores active user sessions for Express session management.

Field	Type	Required	Notes
_id	String	Auto	Session ID (generated by express-session)
userId	ObjectId	Yes	Ref: users._id
expires	Date	Yes	TTL for automatic cleanup
createdAt	Date	Auto	

4. Recommended MongoDB Indexes

The following indexes should be created at startup to ensure query performance on the most common access patterns:

Collection	Index	Reason
users	email (unique)	Login lookup by email
users	username (unique)	Username uniqueness check
apps	ownerId	List all apps for a user
events	appld + type	Dashboard queries by app and event type
events	appld + timestamp	Time-range queries for charting
events	appld + receivedAt	Recency-based retrieval
sessions	expires (TTL)	Automatic session expiry cleanup

5. Open Items — Pending Frontend/UX Input

The following decisions are deferred until the frontend and UX team shares their page and data requirements:

- Exact query parameter names for filtering/pagination on retrieval endpoints
- Whether /app/:id needs sub-routes (e.g. /app/:id/bugs) or if a single endpoint with a type filter suffices
- Response shape for each retrieval endpoint — what fields the dashboard components actually need
- Whether real-time updates (WebSocket / SSE) are needed for the live health view
- Rate limiting strategy for POST /api/collect (per-API-key limits)
- Error aggregation logic — group duplicate errors by fingerprint, or store every occurrence raw?
- Auth token strategy — confirm session cookies are sufficient or if JWT is preferred for the collector client SDK